

## Η υλοποίηση των προσθέσεων και των πολλαπλασιασμών

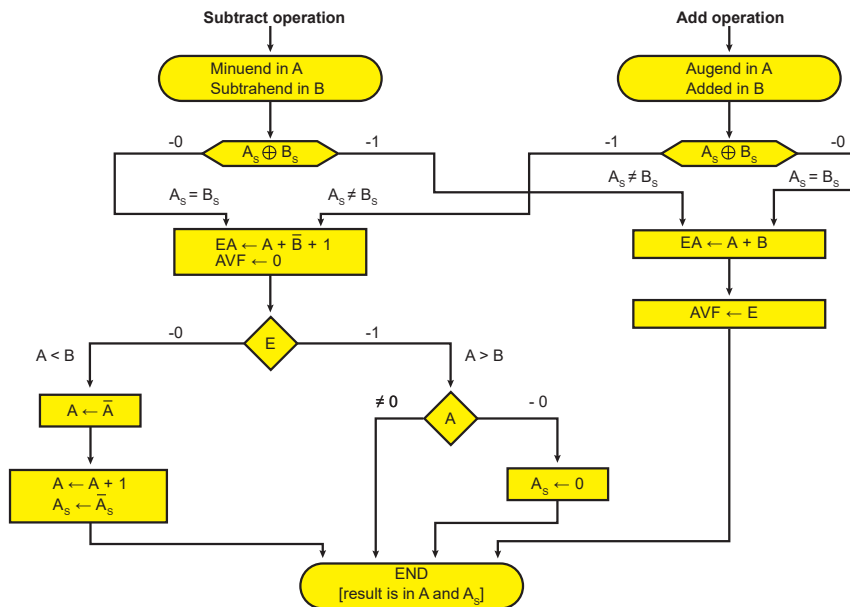
Όπως έχει ήδη αναφερθεί, οι τρεις θεμελιώδεις πράξεις που χρησιμοποιούνται για την υλοποίηση της εξίσωσης διαφορών οποιουδήποτε ψηφιακού φίλτρου, είναι η *πρόσθεση* (η *αφαίρεση*, όπως είναι γνωστό, υλοποιείται μέσω πρόσθεσης), ο *πολλαπλασιασμός* και η *χρονική υστέρηση*. Από αυτές τις πράξεις, η *πιο απλή* στην υλοποίησή της και για την περίπτωση χρήσης σειριακής αριθμητικής είναι η *χρονική υστέρηση* (αφού υλοποιείται πάρα πολύ απλά με τη βοήθεια ενός *καταχωρητή ολίσθησης* με μία απλή είσοδο και μία απλή έξοδο), ενώ η πρόσθεση και ο πολλαπλασιασμός υλοποιούνται με την βοήθεια *απλών κυκλωμάτων* τα οποία είναι γνωστά ως *αθροιστές* και *πολλαπλασιαστές* και τα οποία συνιστούν από κοινού, την *αριθμητική μονάδα* του ψηφιακού φίλτρου. Σε αυτή την ενότητα, παρουσιάζουμε με λεπτομέρεια τις δύο αυτές πράξεις, καθώς και τις δομές μέσω των οποίων αυτές υλοποιούνται. Ειδικότερα, όπως θα σχολιάσουμε στη συνέχεια, αυτές οι δομές υλοποιούνται *τόσο σε σειριακή όσο και σε παράλληλη μορφή*, με την σειριακή μορφή να οδηγεί σε *απλές και οικονομικές λύσεις* και την παράλληλη μορφή να οδηγεί σε *αύξηση της ταχύτητας* και κατά συνέπεια και της *απόδοσης* του φίλτρου.

### Οι πράξεις της πρόσθεσης και της αφαίρεσης

Η πιο απλή αλλά και ταυτόχρονα, θεμελιώδης πράξη που μπορεί να πραγματοποιηθεί ανάμεσα σε δύο αριθμούς εκπεφρασμένους σε οποιοδήποτε αριθμητικό σύστημα, είναι η *πρόσθεση*. Στην περίπτωση του δυαδικού συστήματος που περιέχει μόνο δύο ψηφία, τα πράγματα είναι πολύ απλά, αφού υφίστανται μόνο τέσσερις δυνατότητες και πιο συγκεκριμένα: (α)  $0 + 0 = 0$ , (β)  $0 + 1 = 1$ , (γ)  $1 + 0 = 1$  και (δ)  $1 + 1 = 2$ . Βέβαια, στην τελευταία περίπτωση, ο αριθμός 2 θα πρέπει να διατυπωθεί στο *δυαδικό σύστημα*, δηλαδή να γραφεί ως 10. Επομένως, στην τελευταία περίπτωση θα έχουμε  $1 + 1 = 10$  ή όπως θα λέγαμε στην πρόσθεση που πραγματοποιούμε στο δεκαδικό σύστημα,  $1 + 1 = 0$  και 1 το κρατούμενο. Οι παραπάνω τέσσερις δυνατότητες αφορούν στην περίπτωση πρόσθεσης δύο δυαδικών αριθμών μήκους ενός bit, ενώ στην γενική περίπτωση κατά την οποία επιθυμούμε να προσθέσουμε αριθμούς μήκους  $L$  bits, αναγράφουμε τους αριθμούς τον ένα κάτω από τον άλλο, χαράσσουμε μια οριζόντια γραμμή κάτω από τον δεύτερο αριθμό και στη συνέχεια, προσθέτουμε τα αντίστοιχα bits, με τον ίδιο ακριβώς τρόπο που πραγματοποιούμε την πρόσθεση στο δεκαδικό σύστημα. Εάν οι δύο αριθμοί δεν έχουν το ίδιο μήκος, τότε, πριν την πραγματοποίηση της πρόσθεσης, συνήθως *προσθέτουμε στα αριστερά του αριθμού με το μικρότερο μήκος, το κατάλληλο πλήθος μηδενικών*, έτσι ώστε οι δύο αριθμοί να αποκτήσουν το ίδιο πλήθος από bits. Στην γενική περίπτωση, η πράξη της πρόσθεσης ανάμεσα στα αντίστοιχα bits θα *χρησιμοποιήσει ένα κρατούμενο που έχει προκύψει από την πρόσθεση των bits που βρίσκονται στις προηγούμενες θέσεις των δύο αριθμών και θα δημιουργήσει ένα κρατούμενο που θα χρησιμοποιηθεί στην πρόσθεση των bits που βρίσκονται στις επόμενες θέσεις των δύο αριθμών*, καθώς σαρώνουμε τους αριθμούς από δεξιά προς τα αριστερά, προσθέτοντας τα αντίστοιχα bits. Εάν τουλάχιστον ένα από τα περισσότερα σημαντικά bits των δύο αριθμών έχει τιμή ίση με 1, τότε εάν το κρατούμενο που έχει προκύψει από την προηγούμενη πρόσθεση είναι ίσο με 1, το άθροισμα των δύο αριθμών θα έχει μήκος  $L + 1$  bits, ενώ το ίδιο θα ισχύει και στην περίπτωση κατά την οποία αμφότερα αυτά τα bits έχουν τιμή ίση με 1. Στην περίπτωση αυτή, ενδέχεται να εμφανιστεί το φαινόμενο της *υπερχείλισης* ή όχι, ανάλογα με το μήκος των καταχωρητών του υποκείμενου ψηφιακού συστήματος που υλοποιεί την πράξη της πρόσθεσης.

Η παραπάνω περιγραφή αφορά στην απλή περίπτωση της πρόσθεσης δύο *μη προσημασμένων ακέραιων*, ενώ στην γενική περίπτωση, η πράξη της *πρόσθεσης* πραγματοποιείται με διαφορετικό τρόπο, ανάλογα με τον τύπο της αναπαράστασης που χρησιμοποιείται για την κωδικοποίηση των αριθμών στο δυαδικό σύστημα (θυμηθείτε, πως σε όλους τους τύπους αναπαράστασης, οι θετικοί αριθμοί αναπαρίστανται με τον ίδιο τρόπο, ενώ οι αρνητικοί αριθμοί διαφοροποιούνται ανάλογα με την αναπαράσταση που χρησιμοποιείται). Ειδικότερα, στην *αναπαράσταση πρόσημου μεγέθους*, η κατάσταση είναι *αρκετά περίπλοκη*, αφού υπάρχουν τέσσερις δυνατές περιπτώσεις για την πρόσθεση (που έχουν να κάνουν με το γεγονός, πως για την καθεμία από τις δύο τιμές υπάρχουν δύο εκδοχές όσον αφορά στο πρόσημο) και άλλες τέσσερις περιπτώσεις για την αφαίρεση, αν και αυτές οι οκτώ διαφορετικές εκδοχές μπορούν να μειωθούν, εάν προχωρήσουμε αρχικά στον *έλεγχο των πρόσημων των δύο αριθμών*. Η διαδικασία που πραγματοποιείται απεικονίζεται στο διάγραμμα ροής του Σχήματος 1 και περιλαμβάνει την χρήση της *λογικής πράξης XOR*, προκειμένου να διαπιστώσουμε εάν τα πρόσημα των δύο αριθμών  $A$  και  $B$  είναι *ίδια ή διαφορετικά* (σημειώστε πως εάν αντί για πρόσθεση έχουμε αφαίρεση αυτή υπολογίζεται ως η πρόσθεση  $A - B = A + (-B)$  και κατά συνέπεια, αρχικά θα πρέπει να αλλάξουμε το πρόσημο του δεύτερου αριθμού). Στην πρώτη περίπτωση (κατά την οποία τα πρόσημα είναι τα ίδια), *προσθέτουμε απλά τα μεγέθη των δύο αριθμών και χρησιμοποιούμε ως πρόσημο στο αποτέλεσμα, το κοινό πρόσημο των δύο αριθμών*, ελέγχοντας ταυτόχρονα εάν έχει προκύψει *υπερχείλιση*, ενώ στην δεύτερη περίπτωση (κατά την οποία τα πρόσημα είναι διαφορετικά), *αφαιρούμε τον δεύτερο αριθμό από τον πρώτο* (σημειώστε, πως αντί να εκτελέσουμε την αφαίρεση bit προς bit, προσθέτουμε στον πρώτο αριθμό το συμπλήρωμα ως προς δύο του δεύτερου αριθμού) και χρησιμοποιούμε ως πρόσημο στο αποτέλεσμα, *το πρόσημο του αριθμού με το μεγαλύτερο μέγεθος*. Ανάλογα με την σχέση που υφίσταται ανάμεσα στα μεγέθη των δύο αριθμών, ενδέχεται να εμφανιστεί ή όχι το φαινόμενο της υπερχειλίσης, κατάσταση η οποία θα πρέπει να αντιμετωπιστεί.

Από την άλλη πλευρά, στις *αναπαράστάσεις συμπληρώματος ως προς 1 και ως προς 2*, οι θετικοί αριθμοί αναπαρίστανται όπως και πριν, ενώ για τους αρνητικούς αριθμούς χρησιμοποιείται αντίστοιχα το συμπλήρωμα ως προς 1 και ως προς 2 του αντίστοιχου θετικού αριθμού. Η πρόσθεση των δύο αριθμών πραγματοποιείται ανάμεσα στα αντίστοιχα bits συμπεριλαμβανομένου και του bit πρόσημου και εφόσον η πρόσθεση συνιστά αφαίρεση, αφού, όπως είπαμε, η αφαίρεση  $A - B$  ανάγεται στην πρόσθεση  $A + (-B)$ , διακρίνουμε δύο περιπτώσεις ανάλογα με την σχέση που υφίσταται στα μέτρα των αριθμών  $A$  και  $B$ . Ειδικότερα, στην περίπτωση του συμπληρώματος ως προς 1, εάν είναι  $A > B$ , η αφαίρεση πραγματοποιείται *προσθέτοντας το συμπλήρωμα ως προς 1 του αριθμού  $B$  στον αριθμό  $A$*  και εάν προκύπτει κρατούμενο μετά από το περισσότερο σημαντικό bit, αυτό *προστίθεται* στο αποτέλεσμα, ενώ εάν είναι  $A < B$  η πράξη πραγματοποιείται όπως και στην περίπτωση  $A > B$ , με την διαφορά πως επειδή το αποτέλεσμα της πράξης είναι εκπεφρασμένο στο συμπλήρωμα ως προς 1, *θα πρέπει να λάβουμε το συμπλήρωμα ως*



**Σχήμα 1** Οι διαδικασίες πρόσθεσης και αφαίρεσης ακεραίων που είναι εκπεφρασμένοι στη μορφή πρόσημου μεγέθους (Chauhan P, *Addition and Subtraction*, <https://www.naukri.com/code360/library/addition-and-subtraction>).

προς 1 της τιμής που προέκυψε, με αρνητικό πρόσημο. Αντίθετα, στην περίπτωση της πρόσθεσης δύο θετικών αριθμών που είναι εκπεφρασμένοι στην αναπαράσταση συμπληρώματος ως προς 2, εάν υπάρξει κρατούμενο ύστερα από το bit του πρόσημου αυτό αγνοείται, ενώ εάν το αποτέλεσμα είναι αρνητικό (όπως συμβαίνει για παράδειγμα στην πρόσθεση δύο αρνητικών αριθμών), θα πρέπει να λάβουμε το συμπλήρωμα ως προς 2 της τιμής που προέκυψε. Αντίθετα, στην περίπτωση της αφαίρεσης προσθέτουμε στον πρώτο αριθμό το συμπλήρωμα ως προς 2 του δεύτερου αριθμού.

Προκειμένου να κατανοήσουμε στην πράξη τις παραπάνω αριθμητικές διαδικασίες, ας περιοριστούμε στις αναπαραστάσεις του συμπληρώματος και ας θεωρήσουμε τους αριθμούς  $A = (57)_{10} = (111001)_2$  και  $B = (43)_{10} = (101011)_2$ . Η πράξη της πρόσθεσης δεν παρουσιάζει κάποια δυσκολία και το αποτέλεσμα είναι  $A + B = (57)_{10} + (43)_{10} = (111001)_2 + (101011)_2 = (11001100)_2 = (100)_{10}$ . Για την αφαίρεση  $A - B = 57 - 43$  η οποία θα πραγματοποιηθεί ως  $A + (-B) = 57 + (-43)$  ο αριθμός  $A$  χρησιμοποιείται ως έχει, ενώ για τον αριθμό  $-B = -43$  θα χρησιμοποιηθεί το συμπλήρωμα ως προς 1 του αριθμού  $B = (43)_{10} = (101011)_2$  το οποίο προκύπτει με απλή αντιστροφή των bits και οδηγεί στον δυαδικό αριθμό  $(010100)$ . Κατά συνέπεια, η αφαίρεση  $(111001)_2 - (101011)_2$  ανάγεται στην πρόσθεση  $(111001)_2 + (010100)_2 = (10011101)_2$ . Παρατηρούμε πως το αποτέλεσμα της πρόσθεσης έχει ένα επιπλέον bit σε σχέση με τους δύο αριθμούς (το πρώτο bit από αριστερά), το οποίο απομακρύνεται από εκεί και προστίθεται στο πιο σημαντικό bit του αριθμού που απομένει μετά από αυτή την απομάκρυνση και ο οποίος είναι ο  $(0011101)_2$ . Το αποτέλεσμα αυτής της πρόσθεσης είναι ο αριθμός  $(0011101)_2 + (000001)_2 = (0011110)_2 = (14)_{10}$ , το οποίο, φυσικά, είναι σωστό, αφού στο δεκαδικό σύστημα είναι  $57 - 43 = 14$ . Τέλος, για την αφαίρεση  $B - A = 43 - 57$  η οποία θα πραγματοποιηθεί ως  $B + (-A) = 43 + (-57)$ , ο αριθμός  $B$  χρησιμοποιείται ως έχει, ενώ για τον αριθμό  $-A = -57$  θα χρησιμοποιηθεί το συμπλήρωμα ως προς 1 του αριθμού  $A = (57)_{10} = (111001)_2$  που προκύπτει με απλή αντιστροφή των bits και οδηγεί στον αριθμό  $(0001110)_2$ . Κατά συνέπεια, η αφαίρεση  $(101011)_2 - (111001)_2$  ανάγεται στην πρόσθεση  $(101011)_2 + (0001110)_2 = (110001)_2$ . Παρατηρούμε πως σε αυτό το αποτέλεσμα το πρώτο bit είναι 1 και ως εκ τούτου το αποτέλεσμα είναι αρνητικό. Προκειμένου να προσδιορίσουμε ποιος αριθμός είναι αυτός, θα πρέπει να λάβουμε το συμπλήρωμα ως προς 1 του δυαδικού αριθμού που περιέχει τα υπόλοιπα bits και είναι ο  $(10001)_2$ . Αυτό το συμπλήρωμα που προκύπτει με αντιστροφή των bits είναι ο αριθμός  $(01110)_2 = (14)_{10}$  και κατά συνέπεια το τελικό αποτέλεσμα είναι το  $-14$  που ασφαλώς είναι σωστό.

Με ποιον τρόπο διαφοροποιούνται τα παραπάνω, εάν αντίθετα, χρησιμοποιήσουμε την αναπαράσταση του συμπληρώματος ως προς 2? Στην περίπτωση αυτή, η διαδικασία θα εφαρμοσθεί ως με τον ακόλουθο τρόπο (με τα αποτελέσματα, φυσικά, να είναι τα ίδια). Οι θετικοί αριθμοί  $A = (57)_{10} = (111001)_2$  και  $B = (43)_{10} = (101011)_2$  θα χρησιμοποιηθούν ως έχουν, ενώ για τους αρνητικούς αριθμούς θα χρησιμοποιηθεί το συμπλήρωμα ως προς 2 των αντίθετων θετικών αριθμών, το οποίο προκύπτει από το συμπλήρωμα ως προς 1 προσθέτοντας την μονάδα στο λιγότερο σημαντικό bit. Χρησιμοποιώντας τα αποτελέσματα που κατασκευάσαμε προηγουμένως, ο αριθμός  $-43$  αναπαρίσταται ως  $(010100)_2 + (000001)_2 = (010101)_2$ , ενώ ο αριθμός  $-57$  αναπαρίσταται ως  $(0001110)_2 + (000001)_2 = (0001111)_2$ . Όσον αφορά στις πράξεις, η πρόσθεση θα γίνει ακριβώς όπως και πριν. Η αφαίρεση  $A - B$  θα πραγματοποιηθεί ως  $A + (-B)$  και κατά συνέπεια η αφαίρεση  $(111001)_2 - (101011)_2$  ανάγεται στην πρόσθεση  $(111001)_2 + (010101)_2 = (10011101)_2$ . Παρατηρούμε ότι όπως και πριν, το αποτέλεσμα της πρόσθεσης έχει ένα επιπλέον bit σε σχέση με τους δύο αριθμούς (το πρώτο bit από αριστερά), το οποίο όμως δεν προστίθεται στον αριθμό που αποτελείται από τα υπόλοιπα bits, αλλά απλά αγνοείται. Κατά συνέπεια, το τελικό αποτέλεσμα είναι  $(0011101)_2 = (14)_{10}$ . Με παρόμοιο τρόπο, για την περίπτωση της αφαίρεσης  $B - A = 43 - 57$  η οποία θα πραγματοποιηθεί ως  $B + (-A) = 43 + (-57)$ , ο αριθμός  $B$  χρησιμοποιείται ως έχει, ενώ για τον αριθμό  $-A = -57$  θα χρησιμοποιηθεί το συμπλήρωμα ως προς 1 του αριθμού  $A$  που είναι το  $(0001111)_2$ . Κατά συνέπεια, η αφαίρεση  $(101011)_2 - (111001)_2$  ανάγεται στην πρόσθεση  $(101011)_2 + (0001111)_2 = (110010)_2$ . Παρατηρούμε πως σε αυτό το αποτέλεσμα το πρώτο bit είναι 1 και ως εκ τούτου το αποτέλεσμα είναι αρνητικό.

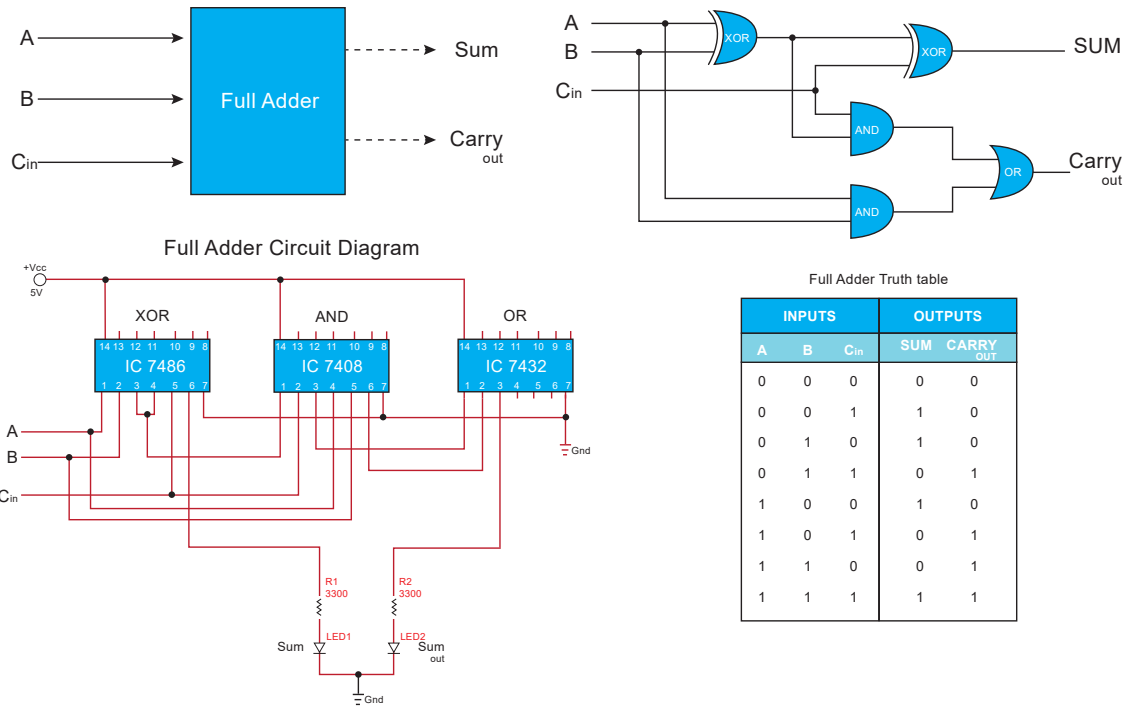
Προκειμένου να προσδιορίσουμε ποιος αριθμός είναι αυτός, θα πρέπει να λάβουμε το συμπλήρωμα ως προς 2 του δυαδικού αριθμού που περιέχει τα υπόλοιπα bits και είναι ο  $(10010)_2$ . Αυτό το συμπλήρωμα που προκύπτει προσθέτοντας στο συμπλήρωμα ως προς 1 που είναι το  $(01101)$  την τιμή 1 έχει τιμή  $(01110)_2 = (14)_{10}$ . Επομένως το αποτέλεσμα της αφαίρεσης στο δεκαδικό σύστημα είναι ο αριθμός  $-14$  που ασφαλώς είναι το ίδιο με πριν.

Η αναπαράσταση του συμπληρώματος ως προς 2, χαρακτηρίζεται από δύο πολύ σημαντικές ιδιότητες όσον αφορά στην πραγματοποίηση της πρόσθεσης, οι οποίες την καθιστούν μία ελκυστική λύση, καθώς και τον κυρίαρχο τρόπο αναπαράστασης αριθμών σε δυαδική μορφή, στους μοντέρνους ηλεκτρονικούς υπολογιστές. Πρώτον, στην περίπτωση της σειριακής πρόσθεσης, η οποία συνίσταται στην πρόσθεση των ζευγών των αντίστοιχων bits των αριθμών που προστίθενται, ένα ζεύγος κάθε φορά, η πράξη της πρόσθεσης μπορεί να πραγματοποιηθεί, ξεκινώντας από την πρόσθεση των λιγότερο σημαντικών bits των δύο αριθμών, χωρίς να γνωρίζουμε ούτε τα πρόσθετα των δύο αριθμών, αλλά και ούτε τη συσχέτιση που υφίσταται ανάμεσά τους όσον αφορά στις τιμές τους. Δεύτερον, κατά την πράξη της πρόσθεσης περισσότερων από δύο αριθμών, εάν η τιμή του σωστού τελικού αθροίσματος είναι τέτοια, ώστε να είναι δυνατή η αναπαράστασή της από το πλήθος των bits που χαρακτηρίζει την αρχιτεκτονική του υποκείμενου ψηφιακού συστήματος, αυτό το άθροισμα εγγυημένα θα υπολογιστεί με τον σωστό τρόπο, ανεξάρτητα από την σειρά με την οποία θα λάβει χώρα η πρόσθεση των αριθμών και μάλιστα, ακόμη και στην περίπτωση, κατά την οποία κατά τον υπολογισμό κάποιων από τα ενδιάμεσα μερικά αθροίσματα, εμφανιστεί το φαινόμενο της υπερχειλίσσης (σημειώστε, πως αυτή η δεύτερη ιδιότητα χαρακτηρίζει και την αναπαράσταση του συμπληρώματος ως προς 1). Αυτό με άλλα λόγια σημαίνει, πως το σύστημα έχει τη δυνατότητα να ανακάμψει από σφάλματα υπερχειλίσσης που έχουν εκδηλωθεί σε προγενέστερες χρονικές στιγμές της λειτουργίας του συστήματος. Η βασική συνέπεια αυτής της ιδιότητας, είναι πως στα συστήματα εκείνα που χρησιμοποιούν την αναπαράσταση του συμπληρώματος ως προς 1, ή την αναπαράσταση του συμπληρώματος ως προς 2, τα μόνα σημεία στα οποία θα πρέπει να διασφαλίσουμε πως δεν θα εμφανιστεί υπερχειλίση, είναι οι είσοδοι των πολλαπλασιαστών και αυτός είναι ο λόγος για τον οποίο, όπως θα δούμε στην οικεία ενότητα, η πράξη της κλιμάκωσης του σήματος που διασφαλίζει την αποτροπή εκδήλωσης φαινομένων υπερχειλίσσης, εφαρμόζεται ακριβώς σε αυτά τα σημεία.

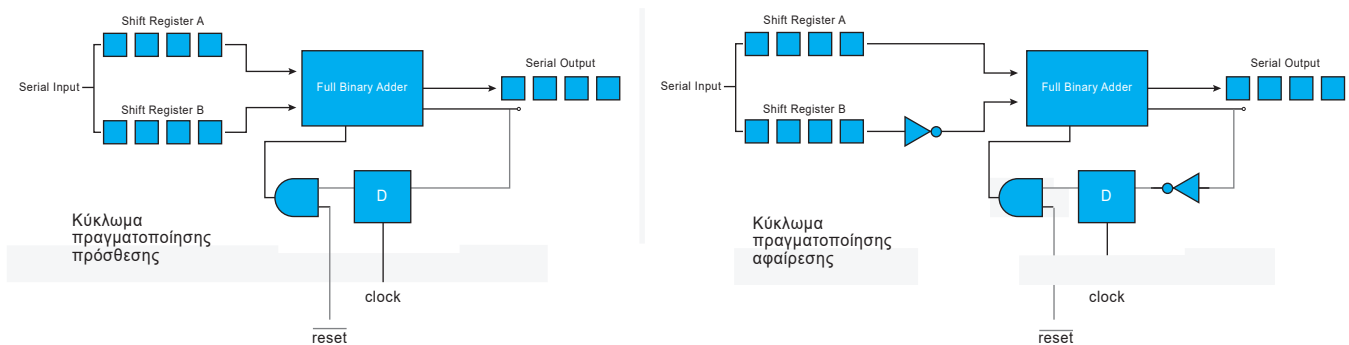
## Σειριακοί και παράλληλοι αθροιστές

Σε επίπεδο υλικού, οι πράξεις της πρόσθεσης και της αφαίρεσης πραγματοποιούνται από ένα ψηφιακό κύκλωμα, το οποίο είναι γνωστό ως αθροιστής και του οποίου οι διάφορες μορφές, απεικονίζονται στο Σχήμα 2. Αυτός ο αθροιστής χαρακτηρίζεται ως πλήρης προκειμένου να διακριθεί από τον ημι-αθροιστή (half adder) ο οποίος δέχεται στην είσοδό του δύο bits τα οποία προσθέτει και εμφανίζει στην έξοδό του, τόσο το αποτέλεσμα της πρόσθεσης, όσο και το bit του κρατούμενου που έχει τιμή ίση με το 0 εάν δεν ανακύπτει κρατούμενο και τιμή με 1, στην αντίθετη περίπτωση (θυμηθείτε πως το κρατούμενο εμφανίζεται μόνο όταν αμφότερα τα bits εισόδου έχουν την τιμή 1). Με άλλα λόγια, το κύκλωμα του ημι-αθροιστή αποτελεί ένα σύστημα δύο εισόδων και δύο εξόδων, το οποίο υλοποιεί απλά τον πίνακα αληθείας της πράξης της δυαδικής πρόσθεσης. Αντίθετα, το κύκλωμα του πλήρους αθροιστή, δέχεται μία επιπλέον είσοδο, η οποία περιέχει το κρατούμενο από την προηγούμενη πρόσθεση, εάν υπάρχει. Με άλλα λόγια, ο πλήρης αθροιστής δέχεται τρεις εισόδους και πιο συγκεκριμένα, τα δύο bits που πρόκειται να προστεθούν μαζί με το κρατούμενο από την προηγούμενη πρόσθεση (που θα έχει την τιμή 0 ή την τιμή 1), καθώς και δύο εξόδους, οι οποίες είναι οι ίδιες με πριν. Στο Σχήμα 2, η επάνω αριστερά εικόνα αποτελεί το σχηματικό διάγραμμα του πλήρους αθροιστή, η επάνω δεξιά εικόνα αποτελεί ένα από τα πολλά ισοδύναμα μεταξύ τους λογικά κυκλώματα του πλήρους αθροιστή, ο οποίος λειτουργεί σύμφωνα με τον πίνακα αληθείας της κάτω δεξιά εικόνας, ενώ η κάτω αριστερά εικόνα, απεικονίζει το αντίστοιχο πραγματικό κυκλωματικό διάγραμμα. Για την δομή των κυκλωμάτων που υλοποιούν τις πύλες που χρησιμοποιούνται σε αυτό, ο αναγνώστης παραπέμπεται στην σχετική πλούσια βιβλιογραφία.

Η πιο απλή περίπτωση κυκλώματος αθροιστή, είναι ο σειριακός αθροιστής, ο οποίος προσθέτει ακολουθιακά τα ζεύγη των αντίστοιχων bits δύο αριθμών, το ένα μετά το άλλο. Αυτό το σύστημα αποτελεί εξαιρετική επιλογή όσον αφορά σε χαρακτηριστικά, όπως είναι η απλότητα της δομής του υλικού και η κατανάλωση ισχύος, αλλά ωστόσο, χαρακτηρίζεται από πολύ μικρή ταχύτητα επεξεργασίας. Αυτός ο αθροιστής δέχεται είσοδο από δύο καταχωρητές ολίσθησης, οι οποίοι περιέχουν τα bits των δύο δυαδικών αριθμών που πρόκειται να προστεθούν, με τα αντίστοιχα bits των δύο αριθμών, να διαβιβάζονται προοδευτικά στις εισόδους του αθροιστή, ξεκινώντας από το λιγότερο σημαντικό bit, προκειμένου να πραγματοποιηθεί η πράξη της πρόσθεσης ανάμεσά τους. Η λειτουργία του συστήματος καθοδηγείται με τη βοήθεια ενός ρολογιού, η περίοδος του οποίου υπαγορεύει τις χρονικές στιγμές αποστολής των ζευγών bits στο κύκλωμα του αθροιστή, το ένα μετά το άλλο. Το bit του κρατούμενου εξόδου που εμφανίζεται στην μία από τις δύο εξόδους του αθροιστή μετά την ολοκλήρωση της πράξης της πρόσθεσης ανάμεσα στο τρέχον ζεύγος των bits που προστίθενται, αποθηκεύεται σε ένα D flip - flop, έτσι ώστε, πριν από την πρόσθεση του επόμενου ζεύγους από bits, να αποσταλεί στην τρίτη είσοδο του αθροιστή, προκειμένου να χρησιμοποιηθεί ως το κρατούμενο εισόδου κατά την πραγματοποίηση της επόμενης πρόσθεσης. Αυτό το bit κρατούμενου, διαβιβάζεται σε μία λογική πύλη AND μαζί με ένα σήμα ελέγχου, το οποίο προκαλεί την αρχικοποίηση του flip - flop δηλαδή ουσιαστικά μηδενίζει το bit του κρατούμενου, όταν ο αθροιστής πρόκειται να ξεκινήσει την πρόσθεση ενός άλλου ζεύγους αριθμών. Αυτό το κύκλωμα του αθροιστή που πραγματοποιεί πρόσθεση συμπληρώματος ως προς 2, απεικονίζεται στην αριστερή εικόνα του Σχήματος 3 ενώ η δεξιά εικόνα απεικονίζει το κύκλωμα που πραγματοποιεί την αφαίρεση δύο αριθμών. Λαμβάνοντας υπόψη πως η αφαίρεση πραγματοποιείται χρησιμοποιώντας για τον αρνητικό αριθμό το συμπλήρωμα ως προς 2, αυτό το κύκλωμα περιέχει μία λογική πύλη NOT που βρίσκεται μετά τον καταχωρητή ολίσθησης ο οποίος περιέχει τα bits του αρνητικού αριθμού, προκαλώντας επομένως, την αντιστροφή τους, ενώ στη συνέχεια, σε αυτή την αντεστραμμένη τιμή που έχει προκύψει, προστίθεται η μονάδα (όπως υπαγορεύεται από τον τρόπο υπολογισμού αυτού του συμπληρώματος), η οποία προέρχεται από μία πύλη NAND, που αντικαθιστά την πύλη AND στο κύκλωμα του αθροιστή. Με τον τρόπο αυτό, κατά την έναρξη της διαδικασίας, το bit του κρατούμενου εισόδου τίθεται στην τιμή 1, η οποία προστίθεται στον αντεστραμμένο αριθμό, έτσι ώστε να υπολογιστεί το συμπλήρωμα ως προς 2. Ωστόσο, αυτή



Σχήμα 2 Το δομικό διάγραμμα του πλήρους αθροιστή μαζί με το λογικό διάγραμμα και το αντίστοιχο λογικό κύκλωμα.



Σχήμα 3 Το δομικό διάγραμμα του σειριακού αθροιστή (αριστερή εικόνα) και οι τροποποιήσεις που πρέπει να γίνουν έτσι ώστε αντί για πρόσθεση να εκτελεί αφαίρεση (δεξιά εικόνα).

η διαδικασία απαιτεί την χρήση και μίας δεύτερης λογικής πύλης NOT, η οποία αντιστρέφει την τιμή του κρατούμενου εξόδου που αποθηκεύεται στο flip - flop. Πράγματι, από την στιγμή που η λογική πύλη NAND, προκαλεί αντιστροφή (ή αναστροφή) τιμής, απαιτείται και μία δεύτερη τέτοια αντιστροφή, προκειμένου να υπολογιστεί και να χρησιμοποιηθεί η σωστή τιμή. Εάν οι αριθμοί δεν αναπαρίστανται σύμφωνα με το συμπλήρωμα ως προς 2, αλλά σύμφωνα με την αναπαράσταση πρόσημου - μεγέθους, ο αθροιστής είναι πιο περίπλοκος, αφού οι κανόνες για την πρόσθεση και την αφαίρεση είναι διαφορετικοί για αυτόν τον τύπο αναπαράστασης και για το λόγο αυτό, συνήθως χρησιμοποιείται ο παραπάνω αθροιστής μαζί με ένα σύστημα μετατροπής της τιμής του αριθμού από την μία αναπαράσταση στην άλλη.

Όπως αναφέραμε προηγουμένως, το βασικό μειονέκτημα του σειριακού αθροιστή, είναι η πολύ μικρή ταχύτητα λειτουργίας του αφού σε κάθε κύκλο του ρολογιού πραγματοποιείται η πρόσθεση ανάμεσα στα αντίστοιχα bits των δύο αριθμών. Για το λόγο αυτό, στην πράξη χρησιμοποιούνται παράλληλοι αθροιστές, οι οποίοι προκύπτουν από την εν σειρά σύνδεση του κατάλληλου πλήθους σειριακών αθροιστών. Για παράδειγμα, συνδέοντας εν σειρά οκτώ σειριακούς αθροιστές του ενός bit, μπορούμε να κατασκευάσουμε έναν παράλληλο αθροιστή των οκτώ bits, ο οποίος μπορεί να υπολογίσει το άθροισμα των αντίστοιχων bits των δύο εισόδων. Αυτή η διαδικασία επιταχύνει σημαντικά την διαδικασία, αφού για έναν αθροιστή αριθμών μήκους  $N$  bits που καθοδηγείται από ένα ρολόι συχνότητας  $f$ , ο παράλληλος αθροιστής που προκύπτει από την εν σειρά σύνδεση  $N$  σειριακών αθροιστών, είναι σε θέση να πραγματοποιεί  $f$  προσθέσεις ανά δευτερόλεπτο, ενώ η αντίστοιχη επίδοση του σειριακού αθροιστή είναι μόλις  $f/N$  προσθέσεις ανά δευτερόλεπτο, με την αύξηση, ωστόσο, της ταχύτητας, να απαιτεί προφανώς και την αύξηση της πολυπλοκότητας του υλικού. Ωστόσο, το βασικό μειονέκτημα του παράλληλου αθροιστή, έχει να κάνει με τον χειρισμό των κρατούμενων. Θυμηθείτε, πως στην καθεμία από τις προσθέσεις που πραγματοποιούνται ανάμεσα στα αντίστοιχα bits των δύο αριθμών που προστίθενται, χρησιμοποιείται ένα κρατούμενο εισόδου (carry-in) ενώ δημιουργείται ένα κρατούμενο εξόδου (carry-out), με τις τιμές αυτών των κρατούμενων, να πρέπει να διατρέχουν όλη τη συστοιχία αυτών των αθροιστών με τέτοιο τρόπο, ώστε το κρατούμενο εξόδου του σειριακού αθροιστή που πραγματοποιεί την πρόσθεση ανάμεσα στο ζεύγος υπ' αριθμόν  $i - 1$  των αντίστοιχων bits, να γίνει κρατούμενο

είσοδου του σειριακού αθροιστή που πραγματοποιεί την πρόσθεση ανάμεσα στο υπ' αριθμόν  $i$  ζεύγος από bits. Αυτή η διαδικασία διάδοσης του κρατούμενου από τη μία βαθμίδα στην επόμενη, είναι αν μη τι άλλο, αρκετά χρονοβόρα και για το λόγο αυτό έχουν καταβληθεί προσπάθειες για την εξομάλυνση αυτού του προβλήματος και γενικότερα την αύξηση της απόδοσης αυτών των βαθμίδων υπολογισμού αθροισμάτων, με προφανές κόστος την αύξηση του βαθμού της πολυπλοκότητας του εμπλεκόμενου υλικού. Μία πρώτη προσπάθεια αφορά στους επονομαζόμενους *αθροιστές διατήρησης κρατούμενου* (carry-save adders), οι οποίοι ανάγουν το πρόβλημα της πρόσθεσης τριών αριθμών μήκους  $k$  bits στο πρόβλημα της πρόσθεσης δύο αριθμών. Αυτές οι διατάξεις, αποτελούν *μία παράλληλη συλλογή από  $k$  το πλήθος πλήρους αθροιστές*, ανάμεσα στους οποίους δεν υφίστανται *οριζόντιες συνδέσεις* (δείτε την αριστερή εικόνα του Σχήματος 4) που δέχονται στην είσοδό τους τρεις ακέραιους αριθμούς  $A$ ,  $B$  και  $C$  και παράγουν στην έξοδό τους δύο ακέραιους αριθμούς  $C'$  και  $S$ , τέτοιους ώστε να είναι  $D + S = A + B + C$ , με το υπ αριθμόν  $i$  bit του αριθμού  $S$  και το υπ' αριθμόν  $i + 1$  bit του αριθμού  $D$  να υπολογίζονται ως  $S_i = A_i \otimes b_i \otimes C_i$  και  $D_{i+1} = A_i B_i + A_i C_i + B_i C_i$ , όπου  $\otimes$  η πράξη της αποκλειστικής λογικής διάζευξης (XOR). Το βασικό χαρακτηριστικό αυτών των διατάξεων είναι πως η *συνολική υστέρηση μετάδοσης, είναι ίση με το άθροισμα των υστερήσεων μετάδοσης τριών πυλών ανεξάρτητα από το πλήθος των bits των αριθμών που προστίθενται*, ενώ το κρατούμενο δεν προωθείται από δεξιά προς τα αριστερά στο ίδιο επίπεδο όπως συμβαίνει στη συνήθη αρχιτεκτονική των παράλληλων αθροιστών αλλά *από πάνω προς τα κάτω*, μέχρι το τελευταίο επίπεδο της πρόσθεσης, διατηρώντας το σε κάθε επίπεδο. Με άλλα λόγια, η κάθε ενδιάμεση πρόσθεση προσθέτει τρεις αριθμούς και υπολογίζει ένα άθροισμα στο οποίο δεν συνυπολογίζονται τα επιμέρους κρατούμενα του ιδίου επιπέδου, αλλά μόνο των προηγούμενων επιπέδων, ενώ τα επιμέρους κρατούμενα του κάθε επιπέδου προωθούνται στο επόμενο επίπεδο. Μία δεύτερη προσπάθεια αφορά στους αθροιστές που χρησιμοποιούν τον *μηχανισμό της διοχέτευσης* (δείτε την δεξιά εικόνα του Σχήματος 4)<sup>1</sup> στους οποίους δεν υφίσταται η *απαιτήση του συγχρονισμού ανάμεσα στα bits μιας λέξης*, ενώ το κρατούμενο κάποιας από τις βαθμίδες της διοχέτευσης, δεν απαιτείται για την λειτουργία της επόμενης βαθμίδας, παρά μόνο κατά την επόμενη χρονοθυρίδα. Μία τρίτη προσπάθεια είναι ο *εκ των προτέρων υπολογισμός όλων των κρατούμενων που χαρακτηρίζουν την πραγματοποίηση μιας πρόσθεσης* (carry look-ahead), τα οποία επομένως είναι όλα γνωστά εξ αρχής και δεν υπάρχει λόγος να μεταδοθούν από τη μία βαθμίδα στην επόμενη. Αυτή η μέθοδος λειτουργεί με τον ακόλουθο τρόπο. Ας συμβολίσουμε με  $x_i$  και  $y_i$  τα υπ' αριθμόν  $i$  bits των δύο αριθμών που επιθυμούμε να προσθέσουμε και έστω πως σε αυτή την πρόσθεση, η οποία θα πραγματοποιηθεί από τον σειριακό αθροιστή υπ' αριθμόν  $i$ , θα χρησιμοποιηθεί το κρατούμενο  $c_i$  που έχει προκύψει κατά την πρόσθεση των bits  $x_{i-1}$  και  $y_{i-1}$ , η οποία έχει πραγματοποιηθεί από τη βαθμίδα υπ' αριθμόν  $i - 1$ . Στην περίπτωση αυτή, καταφεύγοντας στον πίνακα αληθείας του Σχήματος 2 που περιγράφει τον πλήρη αθροιστή, δεν είναι δύσκολο να επαληθεύσουμε, πως οι παραπάνω ποσότητες σχετίζονται μεταξύ τους ως

$$c_i = x_i \cdot y_i + (x_i \otimes y_i) \cdot c_{i-1}$$

με τα σύμβολα  $\cdot$  και  $+$  να περιγράφουν τις πράξεις της λογικής σύζευξης (AND) και της λογικής διάζευξης (OR) και το σύμβολο  $\otimes$  να περιγράφει την λογική πράξη της αποκλειστικής διάζευξης (XOR). Κατασκευάζοντας την παραπάνω εξίσωση για το καθένα από τα  $N - 1$  κρατούμενα που εμπλέκονται στην πρόσθεση δύο δυαδικών αριθμών μήκους  $N$  bits, καταλήγουμε σε ένα σύνολο  $N - 1$  ανεξάρτητων μεταξύ τους εξισώσεων από τις οποίες μπορούμε να υπολογίσουμε τις τιμές των κρατούμενων. Θεωρώντας, για παράδειγμα, έναν παράλληλο αθροιστή των 4 bits, η επαναληπτική εφαρμογή της παραπάνω σχέσης μας δίνει

$$\begin{aligned} c_1 &= x_1 \cdot y_1 \\ c_2 &= x_2 \cdot y_2 + (x_2 \otimes y_2) \cdot c_1 \\ c_3 &= x_3 \cdot y_3 + (x_3 \otimes y_3) \cdot c_2 \end{aligned}$$

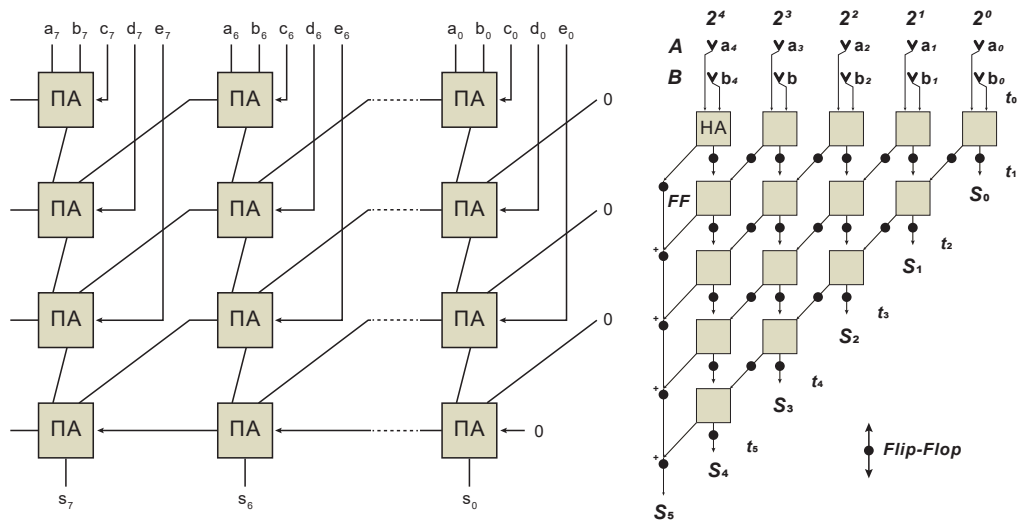
Αν και οι εξισώσεις στην παραπάνω μορφή δεν είναι ιδιαίτερα χρήσιμες, αφού το κρατούμενο  $c_3$  δεν μπορεί να υπολογιστεί χωρίς το  $c_2$  το οποίο με τη σειρά του απαιτεί τον υπολογισμό του  $c_1$ , ωστόσο εάν τις αναδιατάξουμε και τις διατυπώσουμε με τη μορφή

$$\begin{aligned} c_1 &= x_1 \cdot y_1 \\ c_2 &= x_2 \cdot y_2 + (x_2 \otimes y_2) \cdot x_1 \cdot y_1 \\ c_3 &= x_3 \cdot y_3 + (x_3 \otimes y_3) \cdot x_2 \cdot y_2 + (x_3 \otimes y_3) \cdot (x_2 \otimes y_2) \cdot x_1 \cdot y_1 \end{aligned}$$

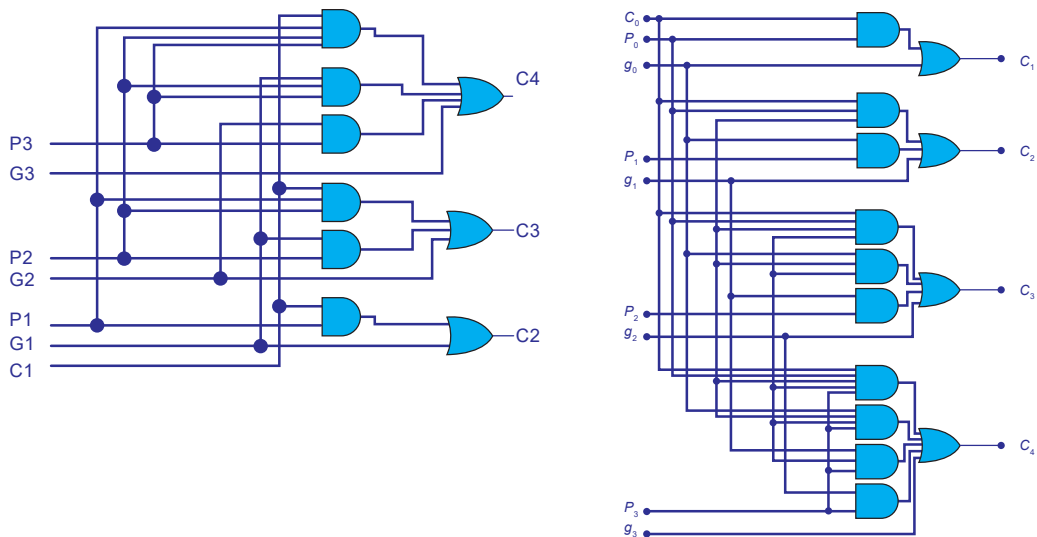
μπορούμε να υπολογίσουμε *εκ των προτέρων* όλα τα κρατούμενα της πράξης της πρόσθεσης από τις τιμές των bits  $x_i$  και  $y_i$  ( $i = 1, 2, \dots, N$ ) τα οποία επομένως, δεν απαιτείται να μεταδοθούν από τον ένα αθροιστή στον επόμενο, όπως συμβαίνει στην παραπάνω υλοποίηση. Ωστόσο, το κόστος αυτής της προσέγγισης, είναι η *αύξηση του βαθμού της πολυπλοκότητας του κυκλώματος*. Εάν απαιτείται η πρόσθεση λέξεων μεγαλύτερου μήκους, τότε η υστέρηση που οφείλεται στην λειτουργία των πυλών δεν αυξάνεται, αλλά ωστόσο, το κόστος του υλικού αυξάνει απότομα, γεγονός που καθιστά αυτή την μέθοδο πρακτικά ανεφάρμοστη για λέξεις μεγάλου μήκους. Αυτή η κατάσταση επιδεικνύεται στο Σχήμα 5, με την αριστερή εικόνα να απεικονίζει το συνδυαστικό κύκλωμα υπολογισμού των κρατούμενων για την πρόσθεση δυαδικών αριθμών μήκους  $N = 3$  bits και την δεξιά εικόνα να απεικονίζει το αντίστοιχο κύκλωμα για την περίπτωση αριθμών μήκους  $N = 4$  bits. Παρατηρήστε πως η αύξηση του μήκους του αριθμού κατά ένα απλό bit, οδήγησε σε αισθητή αύξηση του πλήθους των πυλών και γενικά, της πολυπλοκότητας. Για τον λόγο αυτό, μία ενδιάμεση λύση, είναι ο *διαχωρισμός των λέξεων σε ομάδες λέξεων μικρότερου μήκους* και η μετάδοση των κρατούμενων μόνο ανάμεσα σε αυτές τις ομάδες και όχι για το κάθε bit χωριστά, ενώ στη γενική περίπτωση, ως συνήθως, καταφεύγουμε σε ένα είδος *συμβιβασμού* ανάμεσα στο πρόσθετο υλικό και στις χρονικές υστερήσεις που επάγονται από την χρήση των πυλών.

Τυπικό παράδειγμα παράλληλου αθροιστή που επιτρέπει την πρόσθεση αριθμών μήκους οκτώ bits ο καθένας, απεικονίζεται στο Σχήμα 6. Είναι ενδιαφέρον να σχολιάσουμε σε αυτό το σημείο, πως επειδή στην πρόσθεση των λιγότερο σημαντικών bit,

<sup>1</sup> Για τους αθροιστές αυτού του τύπου δείτε Dadda L & Piuri V, *Pipelined Adders*, IEEE Transactions on Computers, Vol 45, No.3, pp. 348-356, March 1996.



Σχήμα 4 Αθροιστής διατήρησης κρατούμενου (αριστερή εικόνα) και αθροιστής που λειτουργεί με διοχέτευση (δεξιά εικόνα).

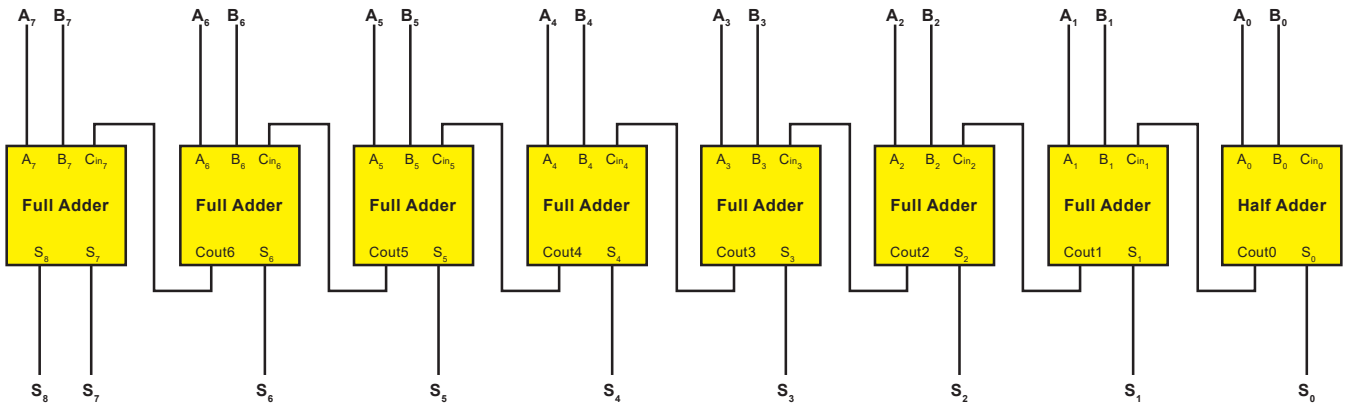


Σχήμα 5 Το συνδυαστικό κύκλωμα που επιτρέπει τον εκ των προτέρων υπολογισμό των κρατούμενων για την πρόσθεση αριθμών με μήκος  $N = 3$  bits (αριστερή εικόνα) και  $N = 4$  bits (δεξιά εικόνα). Οι εισόδους  $p_i$  και  $g_i$  που χρησιμοποιούνται σε αυτά τα κυκλώματα, ορίζονται ως  $p_i = x_i \cdot y_i$  [παραγωγή κρατούμενου (carry generate)] και  $g_i = x_i \otimes y_i$  ( $i = 1, 2, \dots, N$ ) [διάδοση κρατούμενου (carry propagate)].

δεν χρησιμοποιείται κρατούμενο από κάποια προηγούμενη πρόσθεση, αφού αυτή είναι η πρώτη πρόσθεση που πραγματοποιείται, μπορεί αντί για πλήρη αθροιστής να χρησιμοποιηθεί ημι-αθροιστής ο οποίος δεν περιλαμβάνει είσοδο για το κρατούμενο εισόδου. Ωστόσο, συνήθως αυτός ο πρώτος αθροιστής είναι επίσης πλήρης αθροιστής, αφού σε κάποιες αρχιτεκτονικές, όπως στον MIPS η γραμμή του κρατούμενου εισόδου του πρώτου αθροιστή που δεν χρησιμοποιείται, προσφέρει την τιμή 1 που θα προστεθεί στο συμπλήρωμα ως προς 1 του αφαιρέτη μιας πράξης αφαίρεσης, έτσι ώστε να υπολογιστεί το συμπλήρωμα ως προς 2 που θα χρησιμοποιηθεί για την πραγματοποίηση αυτής της πράξης.

### Η πράξη του πολλαπλασιασμού

Χρησιμοποιώντας την πράξη της πρόσθεσης, μπορούμε να πραγματοποιήσουμε την πράξη του πολλαπλασιασμού, η οποία πραγματοποιείται όπως ακριβώς στο δεκαδικό σύστημα. Με άλλα λόγια, πολλαπλασιάζουμε τον πολλαπλασιαστέο με το κάθε ψηφίο του πολλαπλασιαστή, κινούμενοι από δεξιά προς τα αριστερά και στο τέλος της διαδικασίας προσθέτουμε μεταξύ τους τα μερικά γινόμενα τα οποία αναγράφονται μετατοπισμένα κατά μία θέση αριστερά, όπως επιβάλλεται από το γεγονός πως το χρησιμοποιούμενο αριθμητικό σύστημα είναι ένα θεσιακό σύστημα. Αυτή η ολίσθηση κατά ένα bit προς τα αριστερά, πραγματοποιείται με τη βοήθεια ενός flip - flop, ενώ η πρόσθεση των μερικών γινομένων, θα πραγματοποιηθεί ένα γινόμενο τη φορά και όχι σε ένα απλό βήμα, όπως κάνουμε όταν χρησιμοποιούμε χαρτί και μολύβι. Όπως και με την πρόσθεση, τα πράγματα στο δυαδικό σύστημα είναι πολύ πιο απλά σε σχέση με το δεκαδικό σύστημα, αφού υφίστανται μόνο τέσσερις δυνατότητες: (α)  $0 \times 0 = 0$ , (β)  $0 \times 1 = 0$ , (γ)  $1 \times 0 = 0$  και (δ)  $1 \times 1 = 1$ . Παρατηρήστε πως αυτές οι τέσσερις δυνατότητες συλλογικά αποτελούν



Σχήμα 6 Σχηματικό διάγραμμα παράλληλου αθροιστή που επιτρέπει την πρόσθεση δυαδικών αριθμών μήκους οκτώ bits.

$$\begin{array}{r}
 1101 = 13 = \text{data word} \\
 0.101 = 0.625 = \text{coefficient} \\
 \hline
 1101 \\
 0000 \\
 1101 \\
 \hline
 1000.001 = 8.125 = \text{product.}
 \end{array}$$

Σχήμα 7 Παράδειγμα πολλαπλασιασμού στο δυαδικό σύστημα.

τον πίνακα αληθείας της λογικής σύζευξης και κατά συνέπεια ένας πολλαπλασιαστής ενός bit που πραγματοποιεί την πράξη του πολλαπλασιασμού ανάμεσα σε δύο δυαδικούς αριθμούς μήκους ενός bit, δεν είναι παρά μία απλή πύλη AND. Επιπλέον, παρατηρούμε πως για το κάθε μερικό γινόμενο, υφίστανται μόνο δύο δυνατότητες και πιο συγκεκριμένα, να είναι ίσο με το μηδέν, εάν το τρέχον ψηφίο του πολλαπλασιαστή με το οποίο πολλαπλασιάζεται ο πολλαπλασιαστέος είναι ίσο με το 0, ή να είναι ίσο με τον πολλαπλασιαστέο, εφόσον το τρέχον ψηφίο του πολλαπλασιαστή είναι ίσο με 1. Προκειμένου να επιταχυνθεί η διαδικασία, έχουν επινοηθεί εξελιγμένες τεχνικές πολλαπλασιασμού, στις οποίες τα μερικά γινόμενα που αντιστοιχούν σε μηδενική τιμή ψηφίου πολλαπλασιαστή δεν πραγματοποιούνται, αφού η τιμή τους είναι ίση με το μηδέν και το μόνο που γίνεται είναι να πραγματοποιείται η απαιτούμενη ολίσθηση κατά μία θέση αριστερά. Τυπικό παράδειγμα πολλαπλασιασμού ο οποίος πραγματοποιείται στο δυαδικό σύστημα απεικονίζεται στο Σχήμα 7.

Δεν είναι δύσκολο να γίνει αντιληπτό, πως ο τρόπος πραγματοποίησης πολλαπλασιασμών εξαρτάται από τον τύπο της αναπαράστασης που χρησιμοποιείται και ως παρουσιάσουμε στο σημείο αυτό τη διαδικασία, για την περίπτωση του συμπληρώματος ως προς 2 (δείτε Diniz et. al, σελίδα 676). Ας θεωρήσουμε για τον σκοπό αυτό, δύο αριθμούς  $A$  και  $B$  μήκους  $n$  και  $m$  bits αντίστοιχα, οι οποίοι σε αυτόν τον τύπο αναπαράστασης διατυπώνονται ως  $s_A\alpha_1\alpha_2 \cdots \alpha_m$  και  $s_B\beta_1\beta_2 \cdots \beta_n$  ή ισοδύναμα, όπως έχουμε σχολιάσει, ως

$$A = -s_A + \alpha_1 2^{-1} + \alpha_2 2^{-2} + \cdots + \alpha_m 2^{-m} \quad \text{και} \quad B = -s_B + \beta_1 2^{-1} + \beta_2 2^{-2} + \cdots + \beta_n 2^{-n}$$

Καταφεύγοντας στην επιμεριστική ιδιότητα του πολλαπλασιασμού ως προς την πρόσθεση, το γινόμενο  $P = AB$  των δύο αριθμών  $A$  και  $B$ , είναι

$$\begin{aligned}
 P &= (-s_A + \alpha_1 2^{-1} + \alpha_2 2^{-2} + \cdots + \alpha_m 2^{-m})(-s_B + \beta_1 2^{-1} + \beta_2 2^{-2} + \cdots + \beta_n 2^{-n}) \\
 &= (-s_A + \alpha_1 2^{-1} + \alpha_2 2^{-2} + \cdots + \alpha_m 2^{-m})\beta_n 2^{-n} + (-s_A + \alpha_1 2^{-1} + \alpha_2 2^{-2} + \cdots + \alpha_m 2^{-m})\beta_{n-1} 2^{-n+1} + \cdots \\
 &\quad + (-s_A + \alpha_1 2^{-1} + \alpha_2 2^{-2} + \cdots + \alpha_m 2^{-m})\beta_1 2^{-1}(-s_A + \alpha_1 2^{-1} + \alpha_2 2^{-2} + \cdots + \alpha_m 2^{-m})s_B
 \end{aligned}$$

Λαμβάνοντας υπόψη πως το γινόμενο δύο δυαδικών αριθμών  $A$  και  $B$ , είναι ένας δυαδικός αριθμός εκπεφρασμένος ως ένας γραμμικός συνδυασμός δυνάμεων του 2, θα πρέπει να πραγματοποιήσουμε την παραπάνω πράξη με τέτοιο τρόπο, ώστε να είναι δυνατή η διατύπωση του αποτελέσματος στη μορφή ενός τέτοιου γραμμικού συνδυασμού. Αυτό είναι κάτι που μπορεί να γίνει βήμα προς βήμα, πολλαπλασιάζοντας τον αριθμό  $A$  με τον κάθε όρο του αριθμού  $B$  και πραγματοποιώντας τις κατάλληλες παραγοντοποιήσεις. Ειδικότερα, στο πρώτο βήμα πολλαπλασιάζουμε τον αριθμό  $A$  με τους δύο τελευταίους όρους του  $B$  που είναι οι όροι με συντελεστές  $\beta_n$  και  $\beta_{n-1}$ . Θα είναι τότε

$$\begin{aligned}
 C &= \beta_n 2^{-n} A + \beta_{n-1} 2^{-n+1} A \\
 &= (-s_A + \alpha_1 2^{-1} + \alpha_2 2^{-2} + \cdots + \alpha_m 2^{-m})\beta_n 2^{-n} + (-s_A + \alpha_1 2^{-1} + \alpha_2 2^{-2} + \cdots + \alpha_m 2^{-m})\beta_{n-1} 2^{-n+1} \\
 &= 2^{-n+1} [\beta_n 2^{-1}(-s_A + \alpha_1 2^{-1} + \alpha_2 2^{-2} + \cdots + \alpha_m 2^{-m}) + \beta_{n-1}(-s_A + \alpha_1 2^{-1} + \alpha_2 2^{-2} + \cdots + \alpha_m 2^{-m})]
 \end{aligned}$$

Παρατηρούμε τώρα, πως ο πρώτος όρος που βρίσκεται εντός των παραπάνω αγκυλών είναι ο

$$M = \beta_n 2^{-1}(-s_A + \alpha_1 2^{-1} + \alpha_2 2^{-2} + \cdots + \alpha_m 2^{-m}) = \frac{1}{2}\beta_n(-s_A + \alpha_1 2^{-1} + \alpha_2 2^{-2} + \cdots + \alpha_m 2^{-m}) = \frac{N}{2}$$

όπου  $N = \beta_n(-s_A + \alpha_1 2^{-1} + \alpha_2 2^{-2} + \dots + \alpha_m 2^{-m})$  ο οποίος μπορεί να αναπαρασταθεί με τη μορφή

$$M = \beta_n(-s_A + s_A 2^{-1} + \alpha_1 2^{-2} + \alpha_2 2^{-2} + \dots + \alpha_m 2^{-m-1})$$

Αντικαθιστώντας στην παραπάνω έκφραση, θα λάβουμε

$$C = 2^{-n+1} [\beta_n(-s_A + s_A 2^{-1} + \alpha_1 2^{-2} + \alpha_2 2^{-2} + \dots + \alpha_m 2^{-m-1}) + \beta_{n-1}(-s_A + \alpha_1 2^{-1} + \alpha_2 2^{-2} + \dots + \alpha_m 2^{-m})]$$

Πραγματοποιώντας τις πράξεις και ομαδοποιώντας τους όρους που αντιστοιχούν στην ίδια δύναμη του 2, η παραπάνω έκφραση μπορεί να επαναδιατυπωθεί ως

$$C = 2^{-n+2}(-s_C + \gamma_1 2^{-1} + \gamma_2 2^{-2} + \gamma_3 2^{-3} + \dots + \gamma_{m+1} 2^{-m-1} + \gamma_{m+2} 2^{-m-2})$$

με τους συντελεστές  $\gamma_i$  ( $i = 1, 2, \dots, m+2$ ) να εξαρτώνται τόσο από τους συντελεστές  $\alpha_i$  ( $i = 1, 2, \dots, m$ ), όσο και από τους συντελεστές  $\beta_{n-1}$  και  $\beta_n$ , ενώ επειδή το άθροισμα δύο θετικών αριθμών είναι πάντοτε θετικό, ενώ το άθροισμα δύο αρνητικών αριθμών είναι πάντοτε αρνητικό, θα είναι  $s_C = s_A$ .

Στο επόμενο βήμα της διαδικασίας θα πρέπει να προσθέσουμε στον αριθμό  $C$  το γινόμενο  $\beta_{n-2} 2^{-n+2} A$  και εργαζόμενοι με τον ίδιο τρόπο βρίσκουμε

$$\begin{aligned} D &= (-s_A + \gamma_1 2^{-1} + \gamma_2 2^{-2} + \dots + \gamma_{m+2} 2^{-m-2}) 2^{-n+2} + (s_A + \alpha_1 2^{-1} + \alpha_2 2^{-2} + \dots + \alpha_m 2^{-m}) \beta_{n-2} 2^{-n+2} \\ &= 2^{-n+3}(-s_A + \delta_1 2^{-1} + \delta_2 2^{-2} + \dots + \delta_{m+2} 2^{-m-2} + \delta_{m+3} 2^{-m-3}) \end{aligned}$$

Η διαδικασία αυτή συνεχίζεται επαναληπτικά, μέχρι να προστεθούν όλοι οι όροι που εμφανίζονται στο ανάπτυγμα του  $B$  ως γραμμικό συνδυασμό δυνάμεων του 2, πολλαπλασιασμένοι επί  $A$ . Ιδιαίτερη μνεία θα πρέπει να γίνει για τον τελευταίο όρο, ο οποίος θα πρέπει να προστεθεί για την ολοκλήρωση της διαδικασίας, που είναι ο

$$(-s_A + \alpha_1 2^{-1} + \alpha_2 2^{-2} + \dots + \alpha_m 2^{-m}) s_B$$

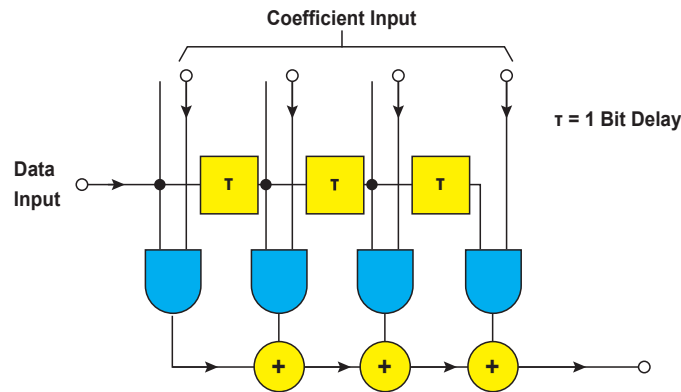
Ειδικότερα, εάν ο αριθμός  $B$  είναι θετικός, η τιμή του  $s_B$  είναι ίση με το μηδέν και η διαδικασία ουσιαστικά έχει ολοκληρωθεί στο προηγούμενο βήμα, ενώ αντίθετα, εάν αυτός ο αριθμός είναι αρνητικός, τότε η τιμή του  $s_B$  είναι ίση με 1 και θα πρέπει να λάβει χώρα η πρόσθεση του όρου

$$-s_A + \alpha_1 2^{-1} + \alpha_2 2^{-2} + \dots + \alpha_m 2^{-m}$$

Αξίζει να σημειωθεί, πως το γινόμενο δύο δυαδικών αριθμών μήκους  $(m+1)$  bits και  $(n+1)$  bits (όπου έχουμε συμπεριλάβει και το bit του πρόσημου), είναι ένας αριθμός μήκους  $m+n+1$  bits και εφόσον επιθυμούμε να αναπαραστήσουμε το τελικό αποτέλεσμα χρησιμοποιώντας μικρότερο πλήθος από bits, θα πρέπει να προχωρήσουμε σε μία διαδικασία κβαντισμού, η οποία μπορεί να πραγματοποιηθεί είτε με περικοπή είτε με στρογγυλοποίηση, όπως περιγράφουμε στην οικεία ενότητα (δείτε την Ενότητα 12-8). Εάν για παράδειγμα, επιθυμούμε να αναπαραστήσουμε το τελικό αποτέλεσμα χρησιμοποιώντας  $m+1$  bits, όσα δηλαδή χρησιμοποιούνται και για την αναπαράσταση του αριθμού  $A$ , θα πρέπει να απομακρύνουμε τα bits που βρίσκονται στις θέσεις  $m+1, m+2, \dots, m+n$ . Στην περίπτωση της περικοπής, το μόνο που έχουμε να κάνουμε είναι να αγνοήσουμε αυτά τα bits, ενώ στην περίπτωση της στρογγυλοποίησης, θα πρέπει πριν από αυτή την περικοπή να προσθέσουμε στο αποτέλεσμα του πολλαπλασιασμού την τιμή  $\Delta = 0.000 \dots 1$ , με το πλήθος των ψηφίων που βρίσκονται μετά την υποδιαστολή, να είναι ίσο με  $m$ , δηλαδή ουσιαστικά, να προσθέσουμε την μονάδα στο bit του γινομένου των δύο αριθμών που βρίσκεται στη θέση  $(m+1)$ , δηλαδή σε εκείνο το bit που απέχει  $n$  θέσεις από το bit υπ αριθμόν  $m+n$ . Στην πραγματικότητα, δεν έχει καμία σημασία εάν αυτό το bit προστεθεί στο τελευταίο ή στο πρώτο άθροισμα της παραπάνω σειράς αθροισμάτων και μία ισοδύναμη διαδικασία για να πραγματοποιήσουμε αυτή τη στρογγυλοποίηση, είναι να προσθέσουμε τον αριθμό  $1.000 \dots 0$ , με το πλήθος των μηδενικών να είναι ίσο με  $n-1$  στο πρώτο μερικό άθροισμα. Σημειώστε επίσης, πως από τη στιγμή που τελικά θα διατηρήσουμε μόνο τα  $m+1$  bits από το τελικό γινόμενο, μπορούμε από το κάθε μερικό άθροισμα να διατηρούμε τα πρώτα  $m+1$  bits.

### Σειριακοί και παράλληλοι πολλαπλασιαστές

Η πραγματοποίηση της πράξης του πολλαπλασιασμού ανάμεσα σε δύο δυαδικούς αριθμούς, πραγματοποιείται με τη βοήθεια των κατάλληλων πολλαπλασιαστών, η δομή και ο τρόπος λειτουργίας των οποίων στη γενική περίπτωση μπορεί να χαρακτηρίζονται από σημαντικές διαφοροποιήσεις. Η πιο απλή τέτοια διάταξη είναι ο επονομαζόμενος σειριακός πολλαπλασιαστής, στον οποίο το κάθε μερικό άθροισμα υπολογίζεται με τη βοήθεια ενός σειριακού αθροιστή, ενώ η στοίχιση των μερικών αθροισμάτων με τον τρόπο που επιβάλλεται από τον τρόπο πραγματοποίησης της πράξης του πολλαπλασιασμού, πραγματοποιείται με την βοήθεια κυκλωμάτων χρονικής υστέρησης. Ένα τυπικό παράδειγμα δομής ενός τέτοιου σειριακού πολλαπλασιαστή που πραγματοποιεί τον πολλαπλασιασμό ανάμεσα σε έναν συντελεστή και σε ένα σήμα δεδομένων, μία κατάσταση που συνεχώς εμφανίζεται στην πράξη, απεικονίζεται στο Σχήμα 8. Παρατηρήστε, πως αν και ο χειρισμός των τιμών των δεδομένων γίνεται με σειριακό τρόπο, ωστόσο, ο χειρισμός των τιμών των συντελεστών πραγματοποιείται με παράλληλο τρόπο και για το λόγο αυτό, ο εν λόγω πολλαπλασιαστής, στην πραγματικότητα χαρακτηρίζεται ως σειριακός - παράλληλος πολλαπλασιαστής. Ωστόσο, παρά αυτόν τον παραλληλισμό, αυτή η διάταξη χαρακτηρίζεται από δύο εγγενή προβλήματα και πιο συγκεκριμένα: (α) η σειριακή σύνδεση των αθροιστών απαιτεί πως κατά τη διάρκεια της κάθε περιόδου bit, τα σήματα διέρχονται από πολλές πύλες, γεγονός που επάγει σημαντικές καθυστερήσεις και (β) αν και στην γενική περίπτωση υφίσταται η απαίτηση τα σήματα εισόδου και εξόδου να έχουν το ίδιο μήκος (αυτή η απαίτηση πηγάζει από το γεγονός, πως οι τιμές των διαδοχικών δειγμάτων είναι παρακείμενες στο χρόνο και δεν



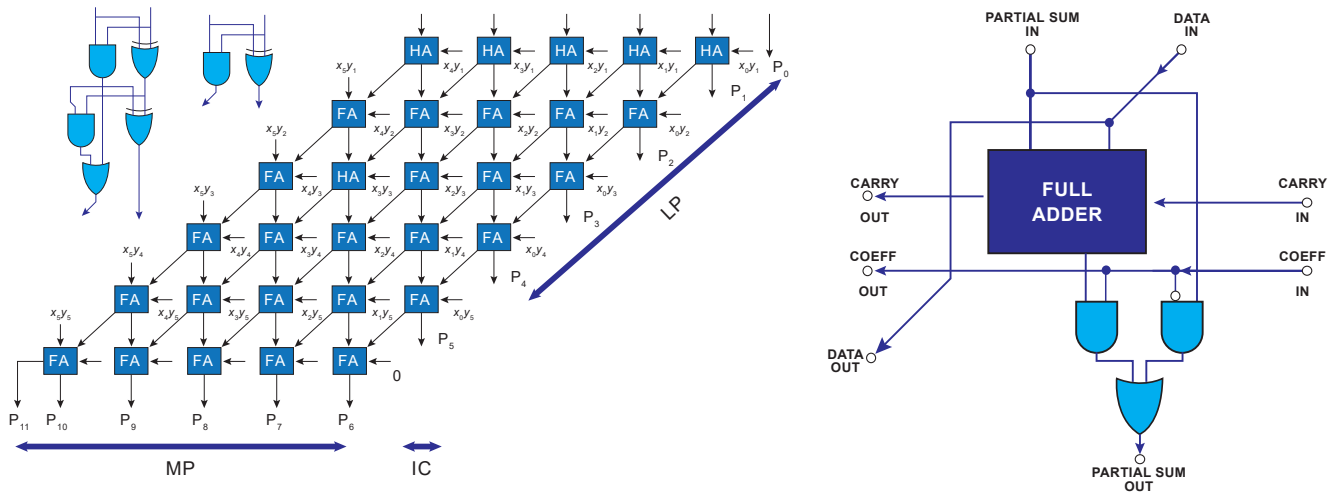
Σχήμα 8 Σχηματικό διάγραμμα σειριακού πολλαπλασιαστή (Freeny, Σχήμα 11).

υπάρχουν διαθέσιμες περισσότερες χρονοθυρίδες για την αποθήκευση μεγαλύτερου πλήθους από bits), δηλαδή οι τιμές τους να κωδικοποιούνται από το ίδιο πλήθος bits, *ωστόσο, αυτό δεν είναι δυνατό, αφού γνωρίζουμε πως το μήκος του γινομένου ισούται με το άθροισμα των μηκών των δύο αριθμών που πολλαπλασιάζονται*. Το πρώτο πρόβλημα επιλύεται καταφεύγοντας ξανά στον μηχανισμό της *διοχέτευσης*, ο οποίος στηρίζεται στην χρήση στοιχείων χρονικής υστέρησης στην κάθε βαθμίδα του σειριακού πολλαπλασιαστή, τα οποία προσφέρουν τη δυνατότητα να ξεκινήσει ο υπολογισμός της τιμής ενός μερικού αθροίσματος, αμέσως μόλις καταστεί διαθέσιμο το πρώτο bit της τιμής του προηγούμενου μερικού αθροίσματος. Αυτή η δυνατότητα, είναι κάτι που αν μη τι άλλο, επιταχύνει σημαντικά την διαδικασία των υπολογισμών. Πράγματι, αν και στην γενική περίπτωση, ο πολλαπλασιασμός δύο αριθμών μήκους  $m + 1$  και  $n + 1$  bits χρησιμοποιώντας έναν σειριακό πολλαπλασιαστή, αποδεικνύεται πως απαιτεί τουλάχιστον  $m + n + 2$  κύκλους του ρολογιού, ακόμη και στην περίπτωση κατά την οποία το τελικό αποτέλεσμα έχει υποστεί κβαντισμό στα  $m + 1$  bits, η χρήση του μηχανισμού της διοχέτευσης, καθιστά δυνατό τον υπολογισμό ενός κβαντισμένου γινομένου μήκους  $m + 1$  bits, σε μόλις  $m + 1$  κύκλους του ρολογιού. Από την άλλη πλευρά, το δεύτερο πρόβλημα επιλύεται προχωρώντας σε *στρογγυλοποίηση της τιμής του γινομένου* με τη βοήθεια του κατάλληλου σήματος. Σημειώστε, πως αν και το παραπάνω κύκλωμα λειτουργεί χρησιμοποιώντας συγκεκριμένες τιμές για τους συντελεστές του φίλτρου, ωστόσο, μπορεί πολύ εύκολα να τροποποιηθεί, έτσι ώστε να υποστηρίζεται η χρήση διαφορετικού συντελεστή για κάθε τιμή της εισόδου, με τις διαφορετικές τιμές των συντελεστών, να διαβάζονται από εξωτερικές μνήμες και να αποθηκεύονται σε καταχωρητές που υφίστανται ειδικά για αυτό το σκοπό. Τέλος, αν και το παραπάνω κύκλωμα λειτουργεί για πολλαπλασιασμό αριθμών εκπεφρασμένων στην *αναπαράσταση πρόσημου - μεγέθους*, ωστόσο μπορεί με μικρές μόνο τροποποιήσεις, να χρησιμοποιηθεί για τον πολλαπλασιασμό αριθμών, η κωδικοποίηση των οποίων στηρίζεται στην αναπαράσταση του *συμπληρώματος ως προς 2*.

Ακόμη μεγαλύτερη επιτάχυνση μπορεί να προκύψει, χρησιμοποιώντας τους επονομαζόμενους *παράλληλους πολλαπλασιαστές*, στους οποίους τα εμπλεκόμενα δεδομένα διακινούνται στο κύκλωμα κατά την *οριζόντια, την κάθετη και την διαγώνια* διεύθυνση, με έναν αποδοτικό τρόπο. Αν και αυτοί οι πολλαπλασιαστές οδηγούν σε *σημαντικότερη επιτάχυνση*, ωστόσο χαρακτηρίζονται από *σχετικά μεγάλο μέγεθος* και από *μεγάλη κατανάλωση ισχύος* και για τον λόγο αυτό χρησιμοποιούνται μόνο σε περιπτώσεις, στις οποίες η ταχύτητα και ο χρόνος επεξεργασίας επηρεάζουν σημαντικά την απόδοση του υποκείμενου ψηφιακού συστήματος. Σε αυτόν τον τύπο του πολλαπλασιαστή, αρχικά πραγματοποιείται ο πολλαπλασιασμός του πολλαπλασιαστέου με το κάθε bit του πολλαπλασιαστή για τον υπολογισμό της τιμής των μερικών γινομένων, τα οποία στη συνέχεια προστίθενται παράλληλα για τον υπολογισμό του τελικού αποτελέσματος. Τυπικό παράδειγμα παράλληλου πολλαπλασιαστή, απεικονίζεται στην αριστερή εικόνα του Σχήματος 9. Στο σχήμα αυτό, το καθένα από τα στοιχεία επεξεργασίας είναι γνωστό ως *κελί Hoffmann* και μία τυπική δομή του, απεικονίζεται στη δεξιά εικόνα του Σχήματος 9. Σε αυτή τη συστοιχία, το πλήθος των γραμμών ισούται με το πλήθος των bits της τιμής του συντελεστή, με το καθένα από αυτά τα bits να ελέγχει και μία γραμμή στοιχείων. Ειδικότερα, εάν η τιμή αυτού του bit είναι ίση με το μηδέν, τότε τα στοιχεία αυτής της γραμμής *προωθούν απλά* το άθροισμα των μερικών γινομένων (που κινούνται κατά την κατακόρυφη διεύθυνση) χωρίς την παραμικρή επεξεργασία, ενώ στην αντίθετη περίπτωση, η τρέχουσα τιμή της εισόδου (που κινείται διαγώνια) *προστίθεται στην κατάλληλη θέση του αθροίσματος των μερικών γινομένων* από τα στοιχεία της γραμμής τα οποία λειτουργούν ως ένας παράλληλος αθροιστής. Αυτή η δομή μπορεί να τροποποιηθεί, έτσι ώστε να πραγματοποιήσει τον κατάλληλο κβαντισμό της τιμής του γινομένου, αλλά και να χρησιμοποιηθεί τόσο με θετικούς όσο και με αρνητικούς αριθμούς.

Είναι ενδιαφέρον να επισημάνουμε, πως εκτός από τις παραπάνω παραδοσιακές προσεγγίσεις υλοποίησης της πράξης του πολλαπλασιασμού, στη βιβλιογραφία μπορούν να βρεθούν και άλλες τεχνικές πραγματοποίησης αυτής της πράξης. Μία προσέγγιση που παρουσιάζει ιδιαίτερο ενδιαφέρον<sup>2</sup>, συνίσταται στην πραγματοποίηση του υπολογισμού των αθροισμάτων των γινομένων ανάμεσα στους συντελεστές του φίλτρου και στα εσωτερικά σήματα του ψηφιακού συστήματος, *χωρίς την ανάγκη φυσικής υλοποίησης δομών πολλαπλασιαστή*. Προκειμένου να επιδείξουμε αυτή την ιδέα, ας θεωρήσουμε ένα σύνολο αριθμών  $\alpha_i$  ( $i = 1, 2, \dots, N$ ) οι οποίοι υποτίθεται πως εκφράζουν *συντελεστές φίλτρου*, καθώς και ένα δεύτερο σύνολο αριθμών  $x_i$  ( $i = 1, 2, \dots, N$ ) που έχουν απεικονισθεί στο διάστημα  $(-1, 1)$ , οι οποίοι υποτίθεται πως εκφράζουν τις τιμές κάποιου εσωτερικού σήματος. Ας υποθέσουμε επίσης, πως αυτά τα δύο σύνολα αριθμών, είναι κωδικοποιημένα στο δυαδικό σύστημα, χρησιμοποιώντας για την κωδικοποίησή τους ένα πλήθος  $K + 1$  bits, εκ των οποίων, ως συνήθως, τα  $K$  bit αναπαριστούν την τιμή του

<sup>2</sup> Δείτε για παράδειγμα Peled A & Liu B, *A new hardware realization of digital filters*, IEEE Transactions on Acoustics, Speech, and Signal Processing, ASSP-22, 456-62, 1974 και Wanhammar L, *An approach to LSI implementation of wave digital filters*, PhD thesis, Linköping University, Linköping, Sweden.



Σχήμα 9 Σχηματικό διάγραμμα παράλληλου πολλαπλασιαστή (αριστερά) και κελί Hoffmann (δεξιά) (Freeny, Σχήμα 15).

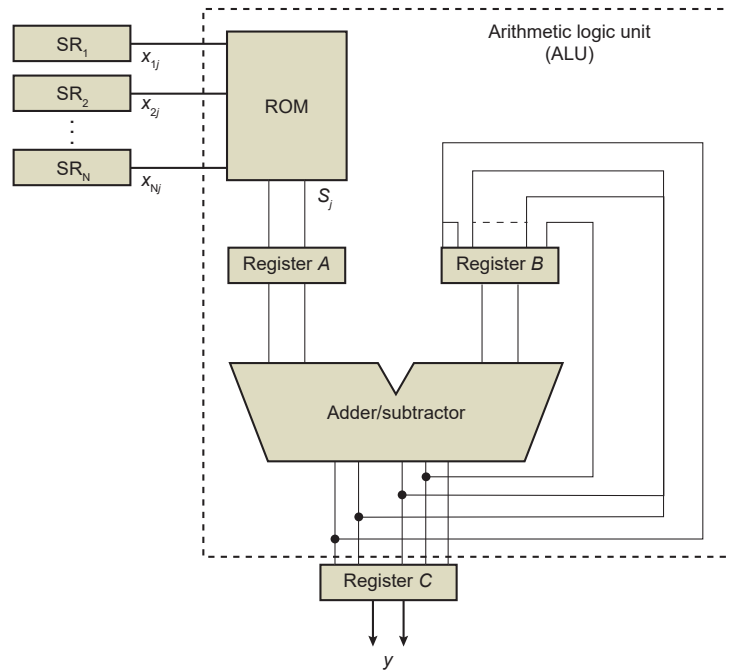
αριθμού, ενώ ένα bit αναπαριστά το πρόσημο. Στην περίπτωση αυτή το άθροισμα των γινομένων  $\alpha_i x_i$  ( $i = 1, 2, \dots, N$ ) μπορεί διαδοχικά να γραφεί ως

$$\begin{aligned} w &= \sum_{i=1}^N \alpha_i x_i = \sum_{i=1}^N \alpha_i \left( \sum_{j=1}^K x_{ij} 2^{-j} - x_{i0} \right) = \sum_{j=1}^K \left( \sum_{i=1}^N \alpha_i x_{ij} \right) 2^{-j} - \sum_{i=1}^N \alpha_i x_{i0} \\ &= \sum_{j=1}^K g(x_{1j}, x_{2j}, \dots, x_{Nj}) 2^{-j} - g(x_{10}, x_{20}, \dots, x_{N0}) = \{[(g_K 2^{-1} + g_{K-1}) 2^{-1} + g_{K-2}] 2^{-1} + \dots + g_1\} 2^{-1} - g_0 \end{aligned}$$

Στην παραπάνω έκφραση, στην οποία χρησιμοποιείται η αναπαράσταση του συμπληρώματος ως προς 2, η ποσότητα  $x_{ij}$  ( $i = 1, 2, \dots, N$  και  $j = 1, 2, \dots, K$ ) εκφράζει το υπ' αριθμόν  $j$  bit του αριθμού  $x_i$ , η ποσότητα  $x_{i0}$  εκφράζει το πρώτο bit αυτού του αριθμού που ως γνωστόν αναπαριστά το πρόσημο, ενώ έχουμε ορίσει

$$g(z_1, z_2, \dots, z_N) = \sum_{i=1}^N \alpha_i z_i$$

Επιπλέον, για λόγους συντομίας της γραφής, χρησιμοποιούμε το συμβολισμό  $g_j = g(x_{1j}, x_{2j}, \dots, x_{Nj})$ . Σύμφωνα με την παραπάνω εξίσωση, μπορούμε να υπολογίσουμε την τιμή του αθροίσματος γινομένων  $w$ , χωρίς να καταφύγουμε στην χρήση ενός συστήματος πολλαπλασιαστή, με τον ακόλουθο τρόπο. Αρχικά υπολογίζουμε την τιμή του  $g_K$  και στη συνέχεια διαιρούμε το αποτέλεσμα με το 2, διαδικασία η οποία πραγματοποιείται πολύ εύκολα και γρήγορα, πραγματοποιώντας δεξιά ολίσθηση της τιμής του  $g_K$  κατά ένα bit. Στη συνέχεια, προσθέτουμε στο αποτέλεσμα που θα προκύψει, την τιμή  $g_{K-1}$ , διαιρούμε το αποτέλεσμα δια 2, προσθέτουμε στην τιμή που θα προκύψει την τιμή του  $g_{K-2}$  και συνεχίζουμε με τον ίδιο τρόπο. Στο τέλος της διαδικασίας και προκειμένου να υπολογίσουμε την τιμή του  $w$ , θα πρέπει να αφαιρέσουμε την τιμή του  $g_0$  από το τελευταίο αποτέλεσμα που θα προκύψει. Το πλεονέκτημα αυτής της μεθόδου έχει να κάνει με το γεγονός πως οι συναρτήσεις  $g(z_1, z_2, \dots, z_N)$  μπορούν να λάβουν το πολύ  $2^N$  διαφορετικές τιμές (αφού οι ποσότητες  $z_i$  ( $i = 1, 2, \dots, N$ ) είναι δυαδικοί αριθμοί του ενός bit και επομένως μπορούν να λάβουν μόνον τις τιμές 0 και 1), οι οποίες εξαρτώνται ασφαλώς και από τις τιμές των συντελεστών  $\alpha_i$  ( $i = 1, 2, \dots, N$ ) και οι οποίες μπορούν να υπολογιστούν εκ των προτέρων και να αποθηκευτούν σε μία μνήμη μόνο ανάγνωσης (ROM), προκειμένου να χρησιμοποιηθούν στους υπολογισμούς. Η παραπάνω διαδικασία μπορεί να πραγματοποιηθεί από ένα κύκλωμα σαν και αυτό που απεικονίζεται στο Σχήμα 10 (δείτε Diniz, σελίδα 687) στο οποίο οι τιμές  $x_1, x_2, \dots, x_N$  αποθηκεύονται στους καταχωρητές ολίσθησης  $R_1, R_2, \dots, R_N$  που έχουν μήκος  $k + 1$  bits και οι  $N$  έξοδοι των οποίων που είναι οι  $x_{1,K-j}, x_{2,K-j}, \dots, x_{N,K-j}$  ορίζουν από κοινού τη διεύθυνση της μνήμης ROM στην οποία βρίσκεται αποθηκευμένη η τιμή της συνάρτησης  $g_{K-j}$ . Μετά την ανάγνωση αυτής της τιμής από τη μνήμη ROM, αυτή αντιγράφεται στον καταχωρητή A προκειμένου να προστεθεί στην τιμή που βρίσκεται ήδη στον καταχωρητή B και η οποία δεν είναι άλλη, από την τρέχουσα συσσωρευτική τιμή του τελικού γινομένου που έχει υπολογιστεί μέχρι αυτό το σημείο. Στη συνέχεια, το άθροισμα που υπολογίζεται, σύμφωνα με τα όσα αναφέραμε προηγουμένως, διαιρείται δια 2. Κατά την εκκίνηση των υπολογισμών, ο καταχωρητής A αρχικοποιείται στην τιμή  $g_K$ , ενώ ο καταχωρητής B, αρχικοποιείται στην μηδενική τιμή. Το τελικό αποτέλεσμα που είναι η τιμή του  $w$ , προκύπτει αφαιρώντας από τελευταίο μερικό αποτέλεσμα που έχει υπολογιστεί, την τιμή  $g_0$  και βρίσκεται αποθηκευμένο στον καταχωρητή C. Η παραπάνω αρχιτεκτονική μπορεί να βελτιωθεί ως προς την απόδοσή της, ενσωματώνοντας σε αυτή χαρακτηριστικά διοχέτευσης ή εναλλακτικά, διαβάζοντας τις τιμές των ποσοτήτων  $g_j$  όχι σειριακά αλλά παράλληλα και χρησιμοποιώντας παράλληλους αθροιστές για τον ταυτόχρονο υπολογισμό των απαιτούμενων μερικών αθροισμάτων. Αποδεικνύεται πως το χρονικό διάστημα που απαιτείται για τον υπολογισμό ενός δείγματος της εξόδου, εξαρτάται αποκλειστικά από το μήκος  $K$  της κάθε λέξης και όχι από το πλήθος  $N$  των μερικών γινομένων που εμπλέκονται στον υπολογισμό της τιμής του  $w$ . Ειδικότερα, αυτό το χρονικό διάστημα αντιστοιχεί σε  $K + 1$  κύκλους του ρολογιού, με την διάρκεια του κάθε κύκλου να ορίζεται ως



Σχήμα 10 Κύκλωμα κατανεμημένου υπολογισμού αθροίσματος γινομένων.

$\max(t_1, t_2)$ , όπου  $t_1$  είναι ο χρόνος που απαιτείται για την προσπέλασης της μνήμης ROM και  $t_2$  ο χρόνος που απαιτείται για την πραγματοποίηση μίας πρόσθεσης ή αφαίρεσης. Από την άλλη πλευρά, το μέγεθος της ROM εξαρτάται προφανώς από το μήκος  $K$  των λέξεων, ενώ η τιμή του  $N$  δεν θα πρέπει να είναι και πάρα πολύ μεγάλη, για να μην έχουμε μεγάλο χρόνο προσπέλασης της μνήμης ROM. Αυτός ο τύπος του κατανεμημένου αριθμητικού υπολογισμού, χρησιμοποιείται για την υλοποίηση πολλών δομών ψηφιακών φίλτρων (δείτε για παράδειγμα Diniz, σελίδες 688-691).